01 LC 704 | Binary Search (Easy)
Problem: Given an array of integers nums which is sorted in ascending order, and an integer target, write a function to search target in nums. If target exists, then return its index. Otherwise, return -1. You must write an algorithm with O(log n) runtime complexity.
Constraints: 1 <= nums.length <= 10^4; -10^4 < nums[i], target < 10^4; All the integers in nums are unique.; nums is sorted in ascending order.

```
class Solution {
public:
  int search(vector<int>& nums, int target) {
    int l = 0, r = nums.size() - 1;
    while (l < r) {
      int mid = (l + r) >> 1;
      if (nums[mid] >= target) {
        r = mid;
      } else {
        l = mid + 1;
      }
    }
    return nums[l] == target ? l : -1;
  }
};
```

06 LC 154 | Find Minimum in Rotated Sorted Array II (Hard)
Problem: Return the minimum element of a rotated nondecreasing array that may contain duplicates. Minimize the number of operations.
Constraints: n == nums.length; 1 <= n <= 5000; -5000 <= nums[i] <= 5000; nums is sorted and rotated between 1 and n times.

```
class Solution {
public:
  int findMin(vector<int>& nums) {
    int l = 0, r = nums.size() - 1;
    while (l < r) {
      int mid = (l + r) >> 1;
      if (nums[mid] > nums[r]) {
        l = mid + 1;
      } else if (nums[mid] == nums[r]) {
        r--;
      } else {
        r = mid;
      }
    }
    return nums[l];
  }
};
```

34 LC 475 | Heaters (Medium)
Problem: Houses and heaters lie on a line. Every heater uses the same radius and covers points within that distance. Return the minimum radius that covers every house.
Constraints: 1 <= houses.length, heaters.length <= 3 * 10^4; 1 <= houses[i], heaters[i] <= 10^9

```
class Solution {
public:
  int findRadius(vector<int>& houses, vector<int>&
    heaters) {
    ranges::sort(heaters);
    int radius = 0;
    for (int house : houses) {
      auto right = ranges::lower_bound(heaters,
        house);
      int distance = INT_MAX;
      if (right != heaters.end()) distance =
        *right - house;
      if (right != heaters.begin()) distance =
        min(distance, house - *prev(right));
      radius = max(radius, distance);
    }
    return radius;
  }
};
```

02 LC 35 | Search Insert Position (Easy)
Problem: Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order. You must write an algorithm with O(log n) runtime complexity.
Constraints: 1 <= nums.length <= 10^4; -10^4 <= nums[i] <= 10^4; nums contains distinct values sorted in ascending order.; -10^4 <= target <= 10^4

```
class Solution {
public:
  int searchInsert(vector<int>& nums, int target)
    {
    return lower_bound(nums.begin(), nums.end(),
      target) - nums.begin();
  }
};
```

05 LC 153 | Find Minimum in Rotated Sorted Array (Medium)
Problem: Return the minimum element of a rotated strictly increasing array with distinct values. Required runtime: O(log n).
Constraints: n == nums.length; 1 <= n <= 5000; -5000 <= nums[i] <= 5000; All the integers of nums are unique.; nums is sorted and rotated between 1 and n times.

```
class Solution {
public:
  int findMin(vector<int>& nums) {
    int l = 0, r = nums.size() - 1;
    while (l < r) {
      int mid = (l + r) >> 1;
      if (nums[mid] > nums.back()) {
        l = mid + 1;
      } else {
        r = mid;
      }
    }
    return nums[l];
  }
};
```

13 LC 875 | Koko Eating Bananas (Medium)
Problem: Each hour Koko chooses one pile and eats up to k bananas from it; she cannot switch piles during that hour. Return the smallest integer speed k that finishes every pile within h hours.
Constraints: 1 <= piles.length <= 10^4; piles.length <= h <= 10^9; 1 <= piles[i] <= 10^9

```
class Solution {
public:
  int minEatingSpeed(vector<int>& piles, int h) {
    int l = 1, r = ranges::max(piles);
    while (l < r) {
      int mid = (l + r) >> 1;
      int s = 0;
      for (int x : piles) {
        s += (x + mid - 1) / mid;
      }
      if (s <= h) {
        r = mid;
      } else {
        l = mid + 1;
      }
    }
    return l;
  }
};
```

03 LC 34 | Find First and Last Position of Element in Sorted Array (Medium)

Problem: Given an array of integers nums sorted in non-decreasing order, find the starting and ending position of a given target value. If target is not found in the array, return [-1, -1]. You must write an algorithm with O(log n) runtime complexity.
Constraints: 0 <= nums.length <= 10^5; -10^9 <= nums[i] <= 10^9; nums is a non-decreasing array.; -10^9 <= target <= 10^9

```
class Solution {
public:
  vector<int> searchRange(vector<int>& nums, int
    target) {
    int l = lower_bound(nums.begin(), nums.end(),
      target) - nums.begin();
    int r = lower_bound(nums.begin(), nums.end(),
      target + 1) - nums.begin();
    if (l == r) {
      return {-1, -1};
    }
    return {l, r - 1};
  }
};
```

38 LC 2300 | Successful Pairs of Spells and Potions (Medium)

Problem: For each spell, count potions whose strength times that spell's strength is at least success. Return counts in spell order; potions are reusable across separate spell queries.
Constraints: n == spells.length; m == potions.length; 1 <= n, m <= 10^5; 1 <= spells[i], potions[i] <= 10^5; 1 <= success <= 10^10

```
class Solution {
public:
  vector<int> successfulPairs(vector<int>& spells,
    vector<int>& potions, long long success) {
    ranges::sort(potions);
    const int m = potions.size();
    vector<int> ans;
    ans.reserve(spells.size());
    for (int v : spells) {
      auto it = ranges::lower_bound(potions,
        static_cast<double>(success) / v);
      ans.push_back(m - static_cast<int>(it -
        potions.begin()));
    }
    return ans;
  }
};
```

43 LC 528 | Random Pick with Weight (Medium)

Problem: Implement Solution(w) and pickIndex(). Return index i with probability w[i]/sum(w). Positive weights describe the required selection probabilities, not values to return.
Constraints: 1 <= w.length <= 10^4; 1 <= w[i] <= 10^5; pickIndex will be called at most 10^4 times.

```
class Solution {
  vector<long long> prefix;
  mt19937 engine{random_device{}()};
public:
  Solution(vector<int>& w) {
    long long sum = 0;
    for (int weight : w) prefix.push_back(sum +=
      weight);
  }
  int pickIndex() {
    long long value =
      uniform_int_distribution<long long>(1,
      prefix.back())(engine);
    return ranges::lower_bound(prefix, value) -
      prefix.begin();
  }
};
```

04 LC 33 | Search in Rotated Sorted Array (Medium)

Problem: nums is a strictly increasing array rotated at an unknown pivot. Return the index of target, or -1 if absent. Required runtime: O(log n).
Constraints: 1 <= nums.length <= 5000; -10^4 <= nums[i] <= 10^4; All values of nums are unique.; nums is an ascending array that is possibly rotated.; -10^4 <= target <= 10^4

```
class Solution {
public:
  int search(vector<int>& nums, int target) {
    int n = nums.size();
    int left = 0, right = n - 1;
    while (left < right) {
      int mid = (left + right) >> 1;
      if (nums[0] <= nums[mid]) {
        if (nums[0] <= target && target <=
          nums[mid])
          right = mid;
        else
          left = mid + 1;
      } else {
        if (nums[mid] < target && target <= nums[n
          - 1])
          left = mid + 1;
        else
          right = mid;
      }
    }
    return nums[left] == target ? left : -1;
  }
};
```

27 LC 378 | Kth Smallest Element in a Sorted Matrix (Medium)

Problem: Given an n x n matrix where each of the rows and columns is sorted in ascending order, return the kth smallest element in the matrix. Note that it is the kth smallest element in the sorted order, not the kth distinct element. You must find a solution with a memory complexity better than O(n^2).
Constraints: n == matrix.length == matrix[i].length; 1 <= n <= 300; -10^9 <= matrix[i][j] <= 10^9; All the rows and columns of matrix are guaranteed to be sorted in non-decreasing order.; 1 <= k <= n^2

```
class Solution {
public:
  int kthSmallest(vector<vector<int>>& matrix, int
    k) {
    int n = matrix.size();
    int left = matrix[0][0], right = matrix[n -
      1][n - 1];
    while (left < right) {
      int mid = left + right >> 1;
      if (check(matrix, mid, k, n)) {
        right = mid;
      } else {
        left = mid + 1;
      }
    }
    return left;
  }
private:
  bool check(vector<vector<int>>& matrix, int mid,
    int k, int n) {
    int count = 0;
    int i = n - 1, j = 0;
    while (i >= 0 && j < n) {
      if (matrix[i][j] <= mid) {
        count += (i + 1);
        ++j;
      } else {
        --i;
      }
    }
    return count >= k;
  }
};
```

07 LC 81 | Search in Rotated Sorted Array II (Medium)

Problem: nums is a nondecreasing array rotated at an unknown pivot and may contain duplicates. Return whether target exists, using as few operations as possible.
Constraints: 1 <= nums.length <= 5000; -10^4 <= nums[i] <= 10^4; nums is guaranteed to be rotated at some pivot.; -10^4 <= target <= 10^4

```
class Solution {
public:
  bool search(vector<int>& nums, int target) {
    int l = 0, r = nums.size() - 1;
    while (l < r) {
      int mid = (l + r) >> 1;
      if (nums[mid] > nums[r]) {
        if (nums[l] <= target && target <=
          nums[mid]) {
          r = mid;
        } else {
          l = mid + 1;
        }
      } else if (nums[mid] < nums[r]) {
        if (nums[mid] < target && target <=
          nums[r]) {
          l = mid + 1;
        } else {
          r = mid;
        }
      } else {
        --r;
      }
    }
    return nums[l] == target;
  }
};
```

15 LC 410 | Split Array Largest Sum (Hard)

Problem: Given an integer array nums and an integer k, split nums into k non-empty subarrays such that the largest sum of any subarray is minimized. Return the minimized largest sum of the split. A subarray is a contiguous part of the array.
Constraints: 1 <= nums.length <= 1000; 0 <= nums[i] <= 10^6; 1 <= k <= min(50, nums.length)

```
class Solution {
public:
  int splitArray(vector<int>& nums, int k) {
    int left = 0, right = 0;
    for (int& x : nums) {
      left = max(left, x);
      right += x;
    }
    auto check = [&](int mx) {
      int s = 1 << 30, cnt = 0;
      for (int& x : nums) {
        s += x;
        if (s > mx) {
          s = x;
          ++cnt;
        }
      }
      return cnt <= k;
    };
    while (left < right) {
      int mid = (left + right) >> 1;
      if (check(mid)) {
        right = mid;
      } else {
        left = mid + 1;
      }
    }
    return left;
  }
};
```

08 LC 74 | Search a 2D Matrix (Medium)

Problem: You are given an m x n integer matrix matrix with the following two properties: - Each row is sorted in non-decreasing order. - The first integer of each row is greater than the last integer of the previous row. Given an integer target, return true if target is in matrix or false otherwise. You must write a solution in O(log(m * n)) time complexity.
Constraints: m == matrix.length; n == matrix[i].length; 1 <= m, n <= 100; -10^4 <= matrix[i][j], target <= 10^4

```
class Solution {
public:
  bool searchMatrix(vector<vector<int>>& matrix,
    int target) {
    int m = matrix.size(), n = matrix[0].size();
    int left = 0, right = m * n - 1;
    while (left < right) {
      int mid = left + right >> 1;
      int x = mid / n, y = mid % n;
      if (matrix[x][y] >= target) {
        right = mid;
      } else {
        left = mid + 1;
      }
    }
    return matrix[left / n][left % n] == target;
  }
};
```

18 LC 1760 | Minimum Limit of Balls in a Bag (Medium)

Problem: A split replaces one bag by two nonempty bags of balls. Using at most maxOperations splits, minimize the maximum balls in any resulting bag; return that minimum.
Constraints: 1 <= nums.length <= 10^5; 1 <= maxOperations, nums[i] <= 10^9

```
class Solution {
public:
  int minimumSize(vector<int>& nums, int
    maxOperations) {
    int l = 1, r = ranges::max(nums);
    while (l < r) {
      int mid = (l + r) >> 1;
      long long s = 0;
      for (int x : nums) {
        s += (x - 1) / mid;
      }
      if (s <= maxOperations) {
        r = mid;
      } else {
        l = mid + 1;
      }
    }
    return l;
  }
};
```

09 LC 240 | Search a 2D Matrix II (Medium)
Problem: Write an efficient algorithm that searches for a value target in an m x n integer matrix matrix. This matrix has the following properties: - Integers in each row are sorted in ascending from left to right. - Integers in each column are sorted in ascending from top to bottom.
Constraints: m == matrix.length; n == matrix[i].length; 1 <= n, m <= 300; -10^9 <= matrix[i][j] <= 10^9; All the integers in each row are sorted in ascending order.; All the integers in each column are sorted in ascending order.; -10^9 <= target <= 10^9

```
class Solution {
public:
  bool searchMatrix(vector<vector<int>>& matrix,
    int target) {
    int m = matrix.size(), n = matrix[0].size();
    int i = m - 1, j = 0;
    while (i >= 0 && j < n) {
      if (matrix[i][j] == target) {
        return true;
      }
      if (matrix[i][j] > target) {
        --i;
      } else {
        ++j;
      }
    }
    return false;
  }
};
```

50 LC 2141 | Maximum Running Time of N Computers (Hard)
Problem: Run n computers simultaneously using batteries with the listed capacities. A battery powers only one computer at a time, but can be swapped instantly at integer times and cannot be recharged. Return the maximum integer runtime for every computer.
Constraints: 1 <= n <= batteries.length <= 10^5; 1 <= batteries[i] <= 10^9

```
class Solution {
public:
  long long maxRunTime(int n, vector<int>&
    batteries) {
    long long l = 0, r = 0;
    for (int x : batteries) {
      r += x;
    }
    while (l < r) {
      long long mid = (l + r + 1) >> 1;
      long long s = 0;
      for (int x : batteries) {
        s += min(1LL * x, mid);
      }
      if (s >= n * mid) {
        l = mid;
      } else {
        r = mid - 1;
      }
    }
    return l;
  }
};
```

10 LC 278 | First Bad Version (Easy)
Problem: Versions 1..n are good until the first bad version; every later version is bad. Using the supplied isBadVersion(version) API, return the first bad version while minimizing calls.
Constraints: 1 <= bad <= n <= 2^31 - 1

```
class Solution {
public:
  int firstBadVersion(int n) {
    int l = 1, r = n;
    while (l < r) {
      int mid = l + (r - l) / 2;
      if (isBadVersion(mid)) {
        r = mid;
      } else {
        l = mid + 1;
      }
    }
    return l;
  }
};
```

31 LC 300 | Longest Increasing Subsequence (Medium)
Problem: Given an integer array nums, return the length of the longest strictly increasing subsequence.
Constraints: 1 <= nums.length <= 2500; -10^4 <= nums[i] <= 10^4

```
class Solution {
public:
  int lengthOfLIS(vector<int>& nums) {
    vector<int> tails;
    for (int x : nums) {
      auto it = ranges::lower_bound(tails, x);
      if (it == tails.end()) tails.push_back(x);
      else *it = x;
    }
    return tails.size();
  }
};
```

48 LC 1838 | Frequency of the Most Frequent Element (Medium)
Problem: The frequency of an element is the number of times it occurs in an array. You are given an integer array nums and an integer k. In one operation, you can choose an index of nums and increment the element at that index by 1. Return the maximum possible frequency of an element after performing at most k operations.
Constraints: 1 <= nums.length <= 10^5; 1 <= nums[i] <= 10^5; 1 <= k <= 10^5

```
class Solution {
public:
  int maxFrequency(vector<int>& nums, int k) {
    ranges::sort(nums);
    int ans = 1;
    long long s = 0;
    for (int i = 1, j = 0; i < nums.size(); ++i) {
      s += 1LL * (nums[i] - nums[i - 1]) * (i -
        j);
      while (s > k) {
        s -= nums[i] - nums[j++];
      }
      ans = max(ans, i - j + 1);
    }
    return ans;
  }
};
```

11 LC 69 | Sqrt(x) (Easy)

Problem: Given a non-negative integer x, return the square root of x rounded down to the nearest integer. The returned integer should be non-negative as well. You must not use any built-in exponent function or operator. - For example, do not use pow(x, 0.5) in c++ or x 0.5 in python.

Constraints: 0 <= x <= 2^31 - 1

```
class Solution {
public:
  int mySqrt(int x) {
    int l = 0, r = x;
    while (l < r) {
      int mid = (l + r + 1ll) >> 1;
      if (mid > x / mid) {
        r = mid - 1;
      } else {
        l = mid;
      }
    }
    return l;
  }
};
```

35 LC 611 | Valid Triangle Number (Medium)

Problem: Given an integer array nums, return the number of triplets chosen from the array that can make triangles if we take them as side lengths of a triangle.

Constraints: 1 <= nums.length <= 1000; 0 <= nums[i] <= 1000

```
class Solution {
public:
  int triangleNumber(vector<int>& nums) {
    ranges::sort(nums);
    int ans = 0, n = nums.size();
    for (int i = 0; i < n - 2; ++i) {
      for (int j = i + 1; j < n - 1; ++j) {
        int sum = nums[i] + nums[j];
        auto it = ranges::lower_bound(nums.begin()
          + j + 1, nums.end(), sum);
        int k = int(it - nums.begin()) - 1;
        ans += max(0, k - j);
      }
    }
    return ans;
  }
};
```

36 LC 167 | Two Sum II - Input Array Is Sorted (Medium)

Problem: In a nondecreasing array numbers, return the one-based indices of two distinct elements summing to target. Exactly one answer exists. Use only constant auxiliary space.

Constraints: 2 <= numbers.length <= 3 * 10^4; -1000 <= numbers[i] <= 1000; numbers is sorted in non-decreasing order.; -1000 <= target <= 1000; The tests are generated such that there is exactly one solution.

```
class Solution {
public:
  vector<int> twoSum(vector<int>& numbers, int
    target) {
    for (int i = 0, n = numbers.size();; ++i) {
      int x = target - numbers[i];
      int j = lower_bound(numbers.begin() + i + 1,
        numbers.end(), x) - numbers.begin();
      if (j < n && numbers[j] == x) {
        return {i + 1, j + 1};
      }
    }
  }
};
```

12 LC 367 | Valid Perfect Square (Easy)

Problem: Given a positive integer num, return true if num is a perfect square or false otherwise. A perfect square is an integer that is the square of an integer. In other words, it is the product of some integer with itself. You must not use any built-in library function, such as sqrt.

Constraints: 1 <= num <= 2^31 - 1

```
class Solution {
public:
  bool isPerfectSquare(int num) {
    int l = 1, r = num;
    while (l < r) {
      int mid = l + (r - l) / 2;
      if (1LL * mid * mid >= num) {
        r = mid;
      } else {
        l = mid + 1;
      }
    }
    return 1LL * l * l == num;
  }
};
```

39 LC 2563 | Count the Number of Fair Pairs (Medium)

Problem: Given a 0-indexed integer array nums of size n and two integers lower and upper, return the number of fair pairs. A pair (i, j) is fair if: - 0 <= i < j < n, and - lower <= nums[i] + nums[j] <= upper

Constraints: 1 <= nums.length <= 10^5; nums.length == n; -10^9 <= nums[i] <= 10^9; -10^9 <= lower <= upper <= 10^9

```
class Solution {
public:
  long long countFairPairs(vector<int>& nums, int
    lower, int upper) {
    long long ans = 0;
    ranges::sort(nums);
    for (int i = 0; i < nums.size(); ++i) {
      auto j = lower_bound(nums.begin() + i + 1,
        nums.end(), lower - nums[i]);
      auto k = lower_bound(nums.begin() + i + 1,
        nums.end(), upper - nums[i] + 1);
      ans += k - j;
    }
    return ans;
  }
};
```

44 LC 729 | My Calendar I (Medium)

Problem: Implement MyCalendar.book(start,end). Accept and store a half-open interval [start,end) only if it does not overlap any previously accepted booking; return whether it was accepted. Touching endpoints do not overlap.

Constraints: 0 <= start < end <= 10^9; At most 1000 calls will be made to book.

```
class MyCalendar {
public:
  MyCalendar() {
  }
  bool book(int startTime, int endTime) {
    auto e = m.lower_bound(startTime + 1);
    if (e != m.end() && e->second < endTime) {
      return false;
    }
    m[endTime] = startTime;
    return true;
  }
private:
  map<int, int> m;
};
```

14 LC 1011 | Capacity To Ship Packages Within D Days (Medium)

Problem: Ship weights in their given order within days days. Each day carries a consecutive group whose total does not exceed capacity. Return the smallest integer capacity that suffices.
Constraints: 1 <= days <= weights.length <= 5 * 10^4; 1 <= weights[i] <= 500

```
class Solution {
public:
  int shipWithinDays(vector<int>& weights, int
    days) {
    int left = 0, right = 0;
    for (auto& w : weights) {
      left = max(left, w);
      right += w;
    }
    auto check = [&](int mx) {
      int ws = 0, cnt = 1;
      for (auto& w : weights) {
        ws += w;
        if (ws > mx) {
          ws = w;
          ++cnt;
        }
      }
      return cnt <= days;
    };
    while (left < right) {
      int mid = (left + right) >> 1;
      if (check(mid)) {
        right = mid;
      } else {
        left = mid + 1;
      }
    }
    return left;
  }
};
```

16 LC 1482 | Minimum Number of Days to Make m Bouquets (Medium)

Problem: Flower i blooms on bloomDay[i]. Each bouquet uses k adjacent flowers, and each flower is used once. Return the earliest day when m bouquets can be made, or -1 if impossible.
Constraints: bloomDay.length == n; 1 <= n <= 10^5; 1 <= bloomDay[i] <= 10^9; 1 <= m <= 10^6; 1 <= k <= n

```
class Solution {
public:
  int minDays(vector<int>& bloomDay, int m, int k)
    {
    int mx = ranges::max(bloomDay);
    int l = 1, r = mx + 1;
    auto check = [&](int days) {
      int cnt = 0, cur = 0;
      for (int x : bloomDay) {
        cur = x <= days ? cur + 1 : 0;
        if (cur == k) {
          cnt++;
          cur = 0;
        }
      }
      return cnt >= m;
    };
    while (l < r) {
      int mid = (l + r) >> 1;
      if (check(mid)) {
        r = mid;
      } else {
        l = mid + 1;
      }
    }
    return l > mx ? -1 : l;
  }
};
```

17 LC 1552 | Magnetic Force Between Two Balls (Medium)

Problem: Place m balls at distinct positions from position. Maximize the minimum distance between any pair of placed balls and return that distance.
Constraints: n == position.length; 2 <= n <= 10^5; 1 <= position[i] <= 10^9; All integers in position are distinct.; 2 <= m <= position.length

```
class Solution {
public:
  int maxDistance(vector<int>& position, int m) {
    ranges::sort(position);
    int l = 1, r = position.back();
    auto count = [&](int f) {
      int prev = position[0];
      int cnt = 1;
      for (int& curr : position) {
        if (curr - prev >= f) {
          prev = curr;
          cnt++;
        }
      }
      return cnt;
    };
    while (l < r) {
      int mid = (l + r + 1) >> 1;
      if (count(mid) >= m) {
        l = mid;
      } else {
        r = mid - 1;
      }
    }
    return l;
  }
};
```

24 LC 1095 | Find in Mountain Array (Hard)

Problem: Using only MountainArray.get(index) and length(), find the smallest index equal to target in a strictly increasing-then-decreasing mountain array, or -1. Use at most 100 get calls.
Constraints: 3 <= mountainArr.length() <= 10^4; 0 <= target <= 10^9; 0 <= mountainArr.get(index) <= 10^9

```
class Solution {
public:
  int findInMountainArray(int target,
    MountainArray& mountainArr) {
    int n = mountainArr.length();
    int l = 0, r = n - 1;
    while (l < r) {
      int mid = (l + r) >> 1;
      if (mountainArr.get(mid) >
        mountainArr.get(mid + 1)) {
        r = mid;
      } else {
        l = mid + 1;
      }
    }
    auto search = [&](int l, int r, int k) -> int
      {
      while (l < r) {
        int mid = (l + r) >> 1;
        if (k * mountainArr.get(mid) >= k *
          target) {
          r = mid;
        } else {
          l = mid + 1;
        }
      }
      return mountainArr.get(l) == target ? l :
        -1;
    };
    int ans = search(0, l, 1);
    return ans == -1 ? search(l + 1, n - 1, -1) :
      ans;
  }
};
```

19 LC 2187 | Minimum Time to Complete Trips (Medium)

Problem: Bus i takes time[i] units per trip and repeats trips immediately; buses operate independently from time zero. Return the minimum elapsed time for all buses together to complete at least totalTrips trips.

Constraints: 1 <= time.length <= 10^5; 1 <= time[i], totalTrips <= 10^7

```
class Solution {
public:
  long long minimumTime(vector<int>& time, int
    totalTrips) {
    int mi = *min_element(time.begin(),
      time.end());
    long long left = 1, right = 1LL * mi *
      totalTrips;
    while (left < right) {
      long long cnt = 0;
      long long mid = (left + right) >> 1;
      for (int v : time) {
        cnt += mid / v;
      }
      if (cnt >= totalTrips) {
        right = mid;
      } else {
        left = mid + 1;
      }
    }
    return left;
  }
};
```

45 LC 1235 | Maximum Profit in Job Scheduling (Hard)

Problem: We have n jobs, where every job is scheduled to be done from startTime[i] to endTime[i], obtaining a profit of profit[i]. You're given the startTime, endTime and profit arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range. If you choose a job that ends at time X you will be able to start another job that starts at time X.

Constraints: 1 <= startTime.length == endTime.length == profit.length <= 5 * 10^4; 1 <= startTime[i] < endTime[i] <= 10^9; 1 <= profit[i] <= 10^4

```
class Solution {
public:
  int jobScheduling(vector<int>& startTime,
    vector<int>& endTime, vector<int>& profit) {
    int n = profit.size();
    vector<tuple<int, int, int>> jobs(n);
    for (int i = 0; i < n; ++i) jobs[i] =
      {endTime[i], startTime[i], profit[i]};
    ranges::sort(jobs);
    vector<int> dp(n + 1);
    for (int i = 0; i < n; ++i) {
      auto [_, s, p] = jobs[i];
      int j = upper_bound(jobs.begin(),
        jobs.begin() + i, s, [&](int x, auto& job)
        -> bool { return x < get<0>(job); }) -
        jobs.begin();
      dp[i + 1] = max(dp[i], dp[j] + p);
    }
    return dp[n];
  }
};
```

20 LC 2226 | Maximum Candies Allocated to K Children (Medium)

Problem: Split candy piles into smaller piles, without merging. Give k children equal positive integer amounts, with each child's share taken from one resulting pile. Candies may remain unused. Return the largest amount per child, or 0.

Constraints: 1 <= candies.length <= 10^5; 1 <= candies[i] <= 10^7; 1 <= k <= 10^12

```
class Solution {
public:
  int maximumCandies(vector<int>& candies, long
    long k) {
    int l = 0, r = ranges::max(candies);
    while (l < r) {
      int mid = (l + r + 1) >> 1;
      long long cnt = 0;
      for (int x : candies) {
        cnt += x / mid;
      }
      if (cnt >= k) {
        l = mid;
      } else {
        r = mid - 1;
      }
    }
    return l;
  }
};
```

47 LC 2080 | Range Frequency Queries (Medium)

Problem: Implement RangeFreqQuery(arr) with query(left,right,value), returning the number of occurrences of value in the inclusive subarray arr[left..right]. The original array is unchanged.

Constraints: 1 <= arr.length <= 10^5; 1 <= arr[i], value <= 10^4; 0 <= left <= right < arr.length; At most 10^5 calls will be made to query

```
class RangeFreqQuery {
public:
  RangeFreqQuery(vector<int>& arr) {
    for (int i = 0; i < arr.size(); ++i) {
      g[arr[i]].push_back(i);
    }
  }
  int query(int left, int right, int value) {
    if (!g.contains(value)) {
      return 0;
    }
    auto& idx = g[value];
    auto l = lower_bound(idx.begin(), idx.end(),
      left);
    auto r = lower_bound(idx.begin(), idx.end(),
      right + 1);
    return r - l;
  }
private:
  unordered_map<int, vector<int>> g;
};
```

21 LC 4 | Median of Two Sorted Arrays (Hard)

Problem: Given two sorted arrays nums1 and nums2 of size m and n respectively, return the median of the two sorted arrays. The overall run time complexity should be O(log (m+n)).

Constraints: nums1.length == m; nums2.length == n; 0 <= m <= 1000; 0 <= n <= 1000; 1 <= m + n <= 2000; -10^6 <= nums1[i], nums2[i] <= 10^6

```
class Solution {
public:
  double findMedianSortedArrays(vector<int>& a,
    vector<int>& b) {
    if (a.size() > b.size()) return
      findMedianSortedArrays(b, a);
    int m = a.size(), n = b.size(), lo = 0, hi =
      m;
    while (lo <= hi) {
      int i = std::midpoint(lo, hi), j = (m + n +
        1) / 2 - i;
      int al = i ? a[i - 1] : INT_MIN;
      int ar = i < m ? a[i] : INT_MAX;
      int bl = j ? b[j - 1] : INT_MIN;
      int br = j < n ? b[j] : INT_MAX;
      if (al <= br && bl <= ar) {
        if ((m + n) % 2) return max(al, bl);
        return (static_cast<double>(max(al, bl)) +
          min(ar, br)) / 2;
      }
      if (al > br) hi = i - 1;
      else lo = i + 1;
    }
    return 0;
  }
};
```

49 LC 1870 | Minimum Speed to Arrive on Time (Medium)

Problem: Travel the train distances in order at one integer speed. All but the final ride must finish before waiting until the next integer hour to depart. Return the minimum speed arriving within hour, or -1 if impossible.

Constraints: n == dist.length; 1 <= n <= 10^5; 1 <= dist[i] <= 10^5; 1 <= hour <= 10^9; There will be at most two digits after the decimal point in hour.

```
class Solution {
public:
  int minSpeedOnTime(vector<int>& dist, double
    hour) {
    if (dist.size() > ceil(hour)) {
      return -1;
    }
    const int m = 1e7;
    int l = 1, r = m + 1;
    int n = dist.size();
    auto check = [&](int v) {
      double s = 0;
      for (int i = 0; i < n; ++i) {
        double t = dist[i] * 1.0 / v;
        s += i == n - 1 ? t : ceil(t);
      }
      return s <= hour;
    };
    while (l < r) {
      int mid = (l + r) >> 1;
      if (check(mid)) {
        r = mid;
      } else {
        l = mid + 1;
      }
    }
    return l > m ? -1 : l;
  }
};
```

22 LC 162 | Find Peak Element (Medium)

Problem: Return the index of any element strictly greater than both neighbors. Treat values outside the array as negative infinity. Adjacent elements differ; required runtime is O(log n).

Constraints: 1 <= nums.length <= 1000; -2^31 <= nums[i] <= 2^31 - 1; nums[i] != nums[i + 1] for all valid i.

```
class Solution {
public:
  int findPeakElement(vector<int>& nums) {
    int left = 0, right = nums.size() - 1;
    while (left < right) {
      int mid = left + right >> 1;
      if (nums[mid] > nums[mid + 1]) {
        right = mid;
      } else {
        left = mid + 1;
      }
    }
    return left;
  }
};
```

23 LC 852 | Peak Index in a Mountain Array (Medium)

Problem: You are given an integer mountain array arr of length n where the values increase to a peak element and then decrease. Return the index of the peak element. Your task is to solve it in O(log(n)) time complexity.

Constraints: 3 <= arr.length <= 10^5; 0 <= arr[i] <= 10^6; arr is guaranteed to be a mountain array.

```
class Solution {
public:
  int peakIndexInMountainArray(vector<int>& arr) {
    int left = 1, right = arr.size() - 2;
    while (left < right) {
      int mid = (left + right) >> 1;
      if (arr[mid] > arr[mid + 1])
        right = mid;
      else
        left = mid + 1;
    }
    return left;
  }
};
```

25 LC 540 | Single Element in a Sorted Array (Medium)

Problem: You are given a sorted array consisting of only integers where every element appears exactly twice, except for one element which appears exactly once. Return the single element that appears only once. Your solution must run in O(log n) time and O(1) space.

Constraints: 1 <= nums.length <= 10^5; 0 <= nums[i] <= 10^5

```
class Solution {
public:
  int singleNonDuplicate(vector<int>& nums) {
    int l = 0, r = nums.size() - 1;
    while (l < r) {
      int mid = (l + r) >> 1;
      if (nums[mid] != nums[mid ^ 1]) {
        r = mid;
      } else {
        l = mid + 1;
      }
    }
    return nums[l];
  }
};
```

26 LC 658 | Find K Closest Elements (Medium)
Problem: Given a sorted integer array arr, two integers k and x, return the k closest integers to x in the array. The result should also be sorted in ascending order. An integer a is closer to x than an integer b if: - |a - x| < |b - x|, or - |a - x| == |b - x| and a < b
Constraints: 1 <= k <= arr.length; 1 <= arr.length <= 10^4; arr is sorted in ascending order.; -10^4 <= arr[i], x <= 10^4

```
class Solution {
public:
  vector<int> findClosestElements(vector<int>&
    arr, int k, int x) {
    int left = 0, right = arr.size() - k;
    while (left < right) {
      int mid = (left + right) >> 1;
      if (x - arr[mid] <= arr[mid + k] - x)
        right = mid;
      else
        left = mid + 1;
    }
    return vector<int>(arr.begin() + left,
      arr.begin() + left + k);
  }
};
```

29 LC 668 | Kth Smallest Number in Multiplication Table (Hard)
Problem: Nearly everyone has used the Multiplication Table. The multiplication table of size m x n is an integer matrix mat where mat[i][j] == i * j (1-indexed). Given three integers m, n, and k, return the kth smallest element in the m x n multiplication table.
Constraints: 1 <= m, n <= 3 * 10^4; 1 <= k <= m * n

```
class Solution {
public:
  int findKthNumber(int m, int n, int k) {
    int left = 1, right = m * n;
    while (left < right) {
      int mid = (left + right) >> 1;
      int cnt = 0;
      for (int i = 1; i <= m; ++i) cnt += min(mid
        / i, n);
      if (cnt >= k)
        right = mid;
      else
        left = mid + 1;
    }
    return left;
  }
};
```

37 LC 209 | Minimum Size Subarray Sum (Medium)
Problem: Return the minimum length of a nonempty contiguous subarray of positive nums whose sum is at least target, or 0 if no such subarray exists.
Constraints: 1 <= target <= 10^9; 1 <= nums.length <= 10^5; 1 <= nums[i] <= 10^4

```
class Solution {
public:
  int minSubArrayLen(int target, vector<int>&
    nums) {
    int l = 0, n = nums.size();
    long long s = 0;
    int ans = n + 1;
    for (int r = 0; r < n; ++r) {
      s += nums[r];
      while (s >= target) {
        ans = min(ans, r - l + 1);
        s -= nums[l++];
      }
    }
    return ans > n ? 0 : ans;
  }
};
```

28 LC 719 | Find K-th Smallest Pair Distance (Hard)
Problem: The distance of a pair of integers a and b is defined as the absolute difference between a and b. Given an integer array nums and an integer k, return the kth smallest distance among all the pairs nums[i] and nums[j] where 0 <= i < j < nums.length.
Constraints: n == nums.length; 2 <= n <= 10^4; 0 <= nums[i] <= 10^6; 1 <= k <= n * (n - 1) / 2

```
class Solution {
public:
  int smallestDistancePair(vector<int>& nums, int
    k) {
    ranges::sort(nums);
    int left = 0, right = nums.back() -
      nums.front();
    while (left < right) {
      int mid = (left + right) >> 1;
      if (count(mid, k, nums) >= k)
        right = mid;
      else
        left = mid + 1;
    }
    return left;
  }
  int count(int dist, int k, vector<int>& nums) {
    int cnt = 0;
    for (int i = 0; i < nums.size(); ++i) {
      int target = nums[i] - dist;
      int j = lower_bound(nums.begin(),
        nums.end(), target) - nums.begin();
      cnt += i - j;
    }
    return cnt;
  }
};
```

42 LC 911 | Online Election (Medium)
Problem: Implement TopVotedCandidate(persons,times). At query time t, count votes cast at or before t and return the leading candidate. Ties favor the candidate receiving the most recent vote among the tied candidates. Query times are at least the first vote time.
Constraints: 1 <= persons.length <= 5000; times.length == persons.length; 0 <= persons[i] < persons.length; 0 <= times[i] <= 10^9; times is sorted in a strictly increasing order.; times[0] <= t <= 10^9; At most 10^4 calls will be made to q.

```
class TopVotedCandidate {
public:
  TopVotedCandidate(vector<int>& persons,
    vector<int>& times) {
    int n = persons.size();
    this->times = times;
    wins.resize(n);
    vector<int> cnt(n);
    int cur = 0;
    for (int i = 0; i < n; ++i) {
      int p = persons[i];
      ++cnt[p];
      if (cnt[cur] <= cnt[p]) {
        cur = p;
      }
      wins[i] = cur;
    }
  }
  int q(int t) {
    int i = upper_bound(times.begin(),
      times.end(), t) - times.begin() - 1;
    return wins[i];
  }
private:
  vector<int> times;
  vector<int> wins;
};
```

30 LC 786 | K-th Smallest Prime Fraction (Hard)

Problem: You are given a sorted integer array arr containing 1 and prime numbers, where all the integers of arr are unique. You are also given an integer k. For every i and j where 0 <= i < j < arr.length, we consider the fraction arr[i] / arr[j]. Return the kth smallest fraction considered. Return your answer as an array of integers of size 2, where answer[0] == arr[i] and answer[1] == arr[j].

Constraints: 2 <= arr.length <= 1000; 1 <= arr[i] <= 3 * 10^4; arr[0] == 1; arr[i] is a prime number for i > 0.; All the numbers of arr are unique and sorted in strictly increasing order.; 1 <= k <= arr.length * (arr.length - 1) / 2

```
class Solution {
public:
  vector<int>
    kthSmallestPrimeFraction(vector<int>& arr, int
    k) {
    using pii = pair<int, int>;
    int n = arr.size();
    auto cmp = [&](const pii& a, const pii& b) {
      return arr[a.first] * arr[b.second] >
        arr[b.first] * arr[a.second];
    };
    priority_queue<pii, vector<pii>,
      decltype(cmp)> pq(cmp);
    for (int i = 1; i < n; ++i) {
      pq.push({0, i});
    }
    for (int i = 1; i < k; ++i) {
      pii f = pq.top();
      pq.pop();
      if (f.first + 1 < f.second) {
        pq.push({f.first + 1, f.second});
      }
    }
    return {arr[pq.top().first],
      arr[pq.top().second]};
  }
};
```

32 LC 354 | Russian Doll Envelopes (Hard)

Problem: You are given a 2D array of integers envelopes where envelopes[i] = [w_i, h_i] represents the width and the height of an envelope. One envelope can fit into another if and only if both the width and height of one envelope are greater than the other envelope's width and height. Return the maximum number of envelopes you can Russian doll (i.e., put one inside the other). Note: You cannot rotate an envelope.

Constraints: 1 <= envelopes.length <= 10^5; envelopes[i].length == 2; 1 <= w_i, h_i <= 10^5

```
class Solution {
public:
  int maxEnvelopes(vector<vector<int>>& envelopes)
    {
    ranges::sort(envelopes, [](const auto& e1,
      const auto& e2) {
      return e1[0] < e2[0] || (e1[0] == e2[0] &&
        e1[1] > e2[1]);
    });
    int n = envelopes.size();
    vector<int> d{envelopes[0][1]};
    for (int i = 1; i < n; ++i) {
      int x = envelopes[i][1];
      if (x > d[d.size() - 1])
        d.push_back(x);
      else {
        int idx = lower_bound(d.begin(), d.end(),
          x) - d.begin();
        if (idx == d.size()) idx = 0;
        d[idx] = x;
      }
    }
    return d.size();
  }
};
```

33 LC 436 | Find Right Interval (Medium)

Problem: You are given an array of intervals, where intervals[i] = [start_i, end_i] and each start_i is unique. The right interval for an interval i is an interval j such that start_j >= end_i and start_j is minimized. Note that i may equal j. Return an array of right interval indices for each interval i. If no right interval exists for interval i, then put -1 at index i.

Constraints: 1 <= intervals.length <= 2 * 10^4; intervals[i].length == 2; -10^6 <= start_i <= end_i <= 10^6; The start point of each interval is unique.

```
class Solution {
public:
  vector<int>
    findRightInterval(vector<vector<int>>&
    intervals) {
    int n = intervals.size();
    vector<pair<int, int>> arr;
    for (int i = 0; i < n; ++i) {
      arr.emplace_back(intervals[i][0], i);
    }
    ranges::sort(arr);
    vector<int> ans;
    for (auto& e : intervals) {
      int j = lower_bound(arr.begin(), arr.end(),
        make_pair(e[1], -1)) - arr.begin();
      ans.push_back(j == n ? -1 : arr[j].second);
    }
    return ans;
  }
};
```

40 LC 981 | Time Based Key-Value Store (Medium)

Problem: Implement TimeMap.set(key,value,timestamp) and get(key,timestamp). get returns the value with the greatest stored timestamp not exceeding the query, or an empty string if none exists. Set timestamps are strictly increasing.

Constraints: 1 <= key.length, value.length <= 100; key and value consist of lowercase English letters and digits.; 1 <= timestamp <= 10^7; All the timestamps timestamp of set are strictly increasing.; At most 2 * 10^5 calls will be made to set and get.

```
class TimeMap {
public:
  TimeMap() {
  }
  void set(string key, string value, int
    timestamp) {
    ktv[key].emplace_back(timestamp, value);
  }
  string get(string key, int timestamp) {
    auto& pairs = ktv[key];
    pair<int, string> p = {timestamp,
      string({127})};
    auto i = upper_bound(pairs.begin(),
      pairs.end(), p);
    return i == pairs.begin() ? "" : (i -
      1)->second;
  }
private:
  unordered_map<string, vector<pair<int, string>>>
    ktv;
};
```

41 LC 1146 | Snapshot Array (Medium)

Problem: Implement SnapshotArray(length), initially all zeros: set(index,val); snap(), returning consecutive snapshot IDs starting at 0; get(index,snap_id), returning the value at that completed snapshot.

Constraints: 1 <= length <= 5 * 10^4; 0 <= index < length; 0 <= val <= 10^9; 0 <= snap_id < (the total number of times we call snap()); At most 5 * 10^4 calls will be made to set, snap, and get.

```
class SnapshotArray {
public:
  SnapshotArray(int length) {
    arr.resize(length);
  }
  void set(int index, int val) {
    arr[index].emplace_back(i, val);
  }
  int snap() {
    return i++;
  }
  int get(int index, int snap_id) {
    auto it = upper_bound(arr[index].begin(),
      arr[index].end(), make_pair(snap_id,
      INT_MAX));
    return it == arr[index].begin() ? 0 :
      prev(it)->second;
  }
private:
  vector<vector<pair<int, int>>> arr;
  int i = 0;
};
```

46 LC 2070 | Most Beautiful Item for Each Query (Medium)

Problem: Each item has a price and beauty. For every budget query, return the greatest beauty among items priced at most that budget, or 0 if none is affordable. Return results in query order.

Constraints: 1 <= items.length, queries.length <= 10^5; items[i].length == 2; 1 <= price_i, beauty_i, queries[j] <= 10^9

```
class Solution {
public:
  vector<int> maximumBeauty(vector<vector<int>>&
    items, vector<int>& queries) {
    ranges::sort(items);
    int n = items.size();
    int m = queries.size();
    vector<int> idx(m);
    iota(idx.begin(), idx.end(), 0);
    ranges::sort(idx, [&](int i, int j) {
      return queries[i] < queries[j];
    });
    int mx = 0, i = 0;
    vector<int> ans(m);
    for (int j : idx) {
      while (i < n && items[i][0] <= queries[j]) {
        mx = max(mx, items[i][1]);
        ++i;
      }
      ans[j] = mx;
    }
    return ans;
  }
};
```